Cours 1 — Introduction aux Technologies Backend

Par Robert DIASSÉ

1. Comprendre le Web : Client, Serveur et HTTP

1.1 Le modèle client–serveur

Le Web fonctionne sur un principe simple :
un client demande, **un serveur répond**.

```
Navigateur (client)
        |
        |  → Requête HTTP
        v
 Serveur backend
        |
        |  ← Réponse HTTP
        v
Navigateur (client)
```

Le navigateur peut être remplacé par :

- une application mobile
- un autre service web
- un script Python/Node
- un logiciel tiers (Postman, client API…)

Le serveur, lui :

- reçoit les requêtes,
- les interprète,
- applique la logique métier,
- interagit avec une base de données,
- renvoie une réponse.

1.2 Qu'est-ce qu'une requête HTTP ?

HTTP est le protocole qui permet au client d'envoyer une demande au serveur.

Une requête contient :

- **une méthode** (GET, POST, PUT, DELETE…)
- **une URL** (ex : `/api/users/5`)
- **des en-têtes (headers)**
- **un corps (body)** parfois (POST/PUT)

Exemple :

```
GET /api/products HTTP/1.1
Host: example.com
Accept: application/json
```

Le serveur analyse la requête et produit une réponse.

1.3 Qu'est-ce qu'une réponse HTTP ?

Une réponse contient :

- **un code de statut** (200, 201, 404, 500…)
- **des headers**
- **un corps** (ex : JSON)

Exemple :

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 4,
  "name": "Chaussures",
  "price": 30000
}
```

1.4 Les principales méthodes HTTP

Méthode	Rôle
GET	Récupérer une ressource
POST	Créer une ressource
PUT	Modifier entièrement une ressource
PATCH	Modifier partiellement
DELETE	Supprimer une ressource

1.5 Où interviennent les bases de données ?

Lorsqu'un utilisateur fait une requête, la plupart du temps le serveur doit lire ou écrire dans une base.

```
Client → Route → Contrôleur → Modèle → Base de données
```

---

2. Structure d'une application backend moderne

Un backend n'est pas un "désordre de fichiers PHP ou Python".
Il suit une **architecture propre**, composée de couches.

2.1 Architecture MVC (la plus répandue)

```
          +-------------+
          |   Modèle    | ← Interaction BD
          +-------------+
                ↑
                |
       +------------------+
       |   Contrôleur     |
       +------------------+
                ↑
                |
          +-------------+
          |    Vue      |
          +-------------+
```

- **Modèle** : représente une table ou une ressource.
- **Contrôleur** : décide quoi faire selon la requête.
- **Vue** : ce qui est renvoyé au client (HTML ou JSON).

Dans une API moderne, la "vue" devient simplement **une réponse JSON**.

2.2 Architecture en services ou couches

Pour les applications plus complexes :

```
Route → Controller → Service → Repository → ORM → Base de données
```

- **Controller** : reçoit la requête
- **Service** : logique métier principale (ex : calculs, règles)
- **Repository** : toutes les requêtes vers la base
- **ORM** : intermédiaire entre objets et tables SQL

2.3 Microservices (architecture avancée)

Un backend peut être découpé en plusieurs petits services indépendants :

```
Users Service
Payment Service
Notification Service
Product Service
```

Chaque service :

- a sa propre logique
- a sa propre base
- communique via API
- peut être maintenu séparément

---

3. API et modes de communication modernes

3.1 Qu'est-ce qu'une API ?

Une API est une porte d'entrée qui permet à des applications de communiquer.

Métaphore simple

L'API est **le serveur dans un restaurant** :

- le client demande
- le serveur transmet à la cuisine
- la cuisine prépare
- le serveur apporte le résultat

Vous n'entrez jamais dans la cuisine → vous appelez l'API.

3.2 API REST (la plus utilisée)

REST utilise :

- les méthodes HTTP
- des URL (appelées **endpoints**)
- des représentations de ressources (souvent JSON)

Exemples d'endpoints REST :

```
GET /api/users
POST /api/users
```

```
GET /api/users/5
DELETE /api/users/3
```

Ressource

Une entité manipulable : `user`, `product`, `article`, `reservation`.

Endpoint

L'URL d'accès à une ressource.

3.3 SOAP (ancien standard)

SOAP est un protocole basé sur XML.

Caractéristiques :

- très strict
- lourd
- utilisé dans les banques, assurances
- nécessite un WSDL (descriptions formelles)

Aujourd'hui, on enseigne SOAP pour culture générale, mais on utilise REST.

3.4 GraphQL

GraphQL permet au client de demander exactement les champs qu'il veut.

Exemple :

```
{
  user(id: 5) {
    name
    email
    reservations {
      date
      status
    }
  }
}
```

Avantages :

- requêtes flexibles
- pas de surchargement (overfetching)
- une seule URL pour toutes les opérations

3.5 WebSockets

Permet la communication en temps réel :

- chat
- tableau de bord temps réel
- notifications instantanées
- suivi de livraison

Il ne fonctionne pas sur requête/réponse comme HTTP classique.

---

4. Bases de données et ORM

4.1 Base SQL vs NoSQL

SQL (relationnelles) :

- MySQL
- MariaDB
- PostgreSQL
- SQLite

Représentées par tables :

```
users
id | name | email | password
```

NoSQL :

- MongoDB
- Redis
- Cassandra

Plus flexibles, mais moins structurées.

4.2 ORM : définition simple

Un ORM (Object Relational Mapping) permet de manipuler la base **comme des objets** plutôt que d'écrire du SQL brut.

Exemple sans ORM :

```
SELECT * FROM users WHERE id = 5;
```

Exemple avec ORM :

```
$user = User::find(5);
```

Avantages :

- moins d'erreurs SQL
- code plus lisible
- validations intégrées
- relations faciles à gérer
- sécurité renforcée

Dans Laravel, l'ORM est **Eloquent**.

---

5. Authentification et sécurité

5.1 Session

Le serveur stocke l'état d'un utilisateur dans un fichier ou une base.

```
Client → Envoie identifiants
Serveur → Vérifie et stocke la session
Client → Reçoit un cookie de session
```

Idéal pour :

sites web classiques, panneaux admin.

5.2 Token (JWT)

Le serveur génère un **jeton signé**.
Le client le renvoie dans chaque requête.

Avantages :

- idéal pour les API
- stateless (le serveur ne stocke rien)
- compatible mobile / web / desktop

Structure d'un JWT :

```
HEADER.PAYLOAD.SIGNATURE
```

5.3 OAuth2

Standard utilisé par :

- Google Login
- Facebook Login
- GitHub Login

---

6. Panorama des technologies backend

Voici un résumé clair des grandes technologies utilisées aujourd'hui.

PHP

- Laravel (moderne, complet, très utilisé)
- Symfony
- CodeIgniter

JavaScript

- Node.js
- Express
- NestJS

Python

- Django
- Flask
- FastAPI

Java

- Spring Boot
- Quarkus

C#

- ASP.NET Core

Chaque technologie a :

- un écosystème
- un marché d'emploi
- un type de projets adapté

Laravel reste idéal pour commencer car il est :

- simple
- structuré
- pédagogique
- largement utilisé en Afrique et dans le monde

7. Gestionnaires de paquets : outils indispensables

7.1 PHP → Composer

Composer permet :

- d'installer des bibliothèques
- d'activer l'autoload automatique
- de gérer les dépendances
- de versionner proprement les projets

Exemple :

```
composer require phpmailer/phpmailer
```

7.2 JavaScript → npm / yarn / pnpm

Exemple :

```
npm install express
```

7.3 Python → pip

Exemple :

```
pip install django
```

8. Conclusion

Ce cours vous donne les fondations indispensables pour comprendre :

- comment fonctionne le web
- comment les serveurs traitent les requêtes
- ce qu'est une API
- comment sont structurés les backends modernes
- pourquoi les ORM sont essentiels
- pourquoi il faut maîtriser le langage AVANT son framework
- quelles technologies existent
- comment fonctionne l'authentification

- l’importance des gestionnaires de paquets